

LAB MANUAL – Experiment 2

LINEAR REGRESSION USING PYTHON

1. Aim

To implement Simple Linear Regression using Python and understand how a linear model can be used to predict a dependent variable from an independent variable.

2. Learning Objectives

- Understand the concept of linear regression.
- Identify independent and dependent variables.
- Understand the equation of a straight-line regression model.
- Implement linear regression using NumPy.
- Implement linear regression using Scikit-learn.
- Calculate regression performance metrics.
- Visualize the regression line.
- Make predictions using the trained model.

3. Description

Linear Regression is a supervised machine learning algorithm used to predict a continuous numerical value. It assumes that there is a linear relationship between the input variable and the output variable.

For Simple Linear Regression:

$$y = mx + c$$

- x = independent/input variable
- y = dependent/output variable
- m = slope/coefficient
- c = intercept

In machine learning notation: $\hat{y} = \beta_0 + \beta_1 x$

Mean Squared Error (MSE)

$$MSE = (1/n) \sum (y - \hat{y})^2$$

Gives greater importance to larger errors because errors are squared. Lower MSE is better.

Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

Expressed in the same unit as the target variable. Lower RMSE generally indicates a better model.

R² Score

$$R^2 = 1 - SS_{res}/SS_{tot}$$

Indicates how much of the variation in the dependent variable is explained by the regression model.

Generally: $R^2 = 1$ indicates perfect prediction; $R^2 = 0$ means the model explains none of the variation; higher R^2 generally indicates a better fit.

Activity #1 With only NumPy

Suppose we want to predict a student's marks based on the number of hours studied.

Hours Studied	Marks
1	35
2	40
3	50
4	55
5	65
6	70
7	75
8	85

Input (X) = Hours Studied; Output (y) = Marks. The regression model attempts to find the best line: Marks = $m(\text{Hours}) + c$.

1A. Dataset

```
import numpy as np

hours = np.array([1, 2, 3, 4, 5, 6, 7, 8])
marks = np.array([35, 40, 50, 55, 65, 70, 75, 85])
```

1B. Exploring the Dataset

Step 1: Display the Data

```
print("Hours:", hours)
print("Marks:", marks)
```

Expected Output:

```
Hours: [1 2 3 4 5 6 7 8]
Marks: [35 40 50 55 65 70 75 85]
```

Step 2: Calculate Basic Statistics

Calculate the mean of hours, mean of marks, standard deviation of marks, minimum marks, and maximum marks.

```
print("Mean Hours:", np.mean(hours))
print("Mean Marks:", np.mean(marks))
print("Standard Deviation:", np.std(marks))
print("Minimum Marks:", np.min(marks))
print("Maximum Marks:", np.max(marks))
```

1C. Visualizing the Relationship

Step 3: Create a Scatter Plot

```
import matplotlib.pyplot as plt

plt.scatter(hours, marks)
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Hours Studied vs Marks")
plt.show()
```

1D. Linear Regression Using NumPy

Step 4: Find the Regression Equation

NumPy provides the `polyfit()` function for fitting a polynomial. For linear regression, the degree is 1.

```
m, c = np.polyfit(hours, marks, 1)
print("Slope:", m)
print("Intercept:", c)
```

The resulting equation has the form: $\hat{y} = mx + c$.

Step 5: Calculate Predictions

```
predicted_marks = m * hours + c
print("Predicted Marks:")
print(predicted_marks)
```

Step 6: Predict Marks for a New Student

```
new_hours = 6.5
predicted = m * new_hours + c
print("Predicted Marks:", predicted)
```

Task: Modify the program to predict marks for 2.5, 4.5, 7.5, and 10 hours.

1E. Plot the Regression Line

Step 7: Visualize Actual Data and Regression Line

```
plt.scatter(hours, marks, label="Actual Data")
plt.plot(hours, predicted_marks, label="Regression Line")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Linear Regression")
plt.legend()
plt.show()
```

Observation: The scatter points represent actual observations and the straight line represents the values predicted by the regression model.

1F. Calculate Errors

Error = Actual – Predicted

```
errors = marks - predicted_marks
print("Errors:")
print(errors)
```

$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$

```
mse = np.mean((marks - predicted_marks) ** 2)
print("Mean Squared Error:", mse)
```

RMSE = $\sqrt{MSE}$

```
rmse = np.sqrt(mse)
print("RMSE:", rmse)
```

Interpretation: RMSE represents the typical magnitude of prediction error in the same unit as the target variable, here marks.

Activity #2: With Sklearn function

Step 1: Import LinearRegression

```
from sklearn.linear_model import LinearRegression
```

Step 2: Reshape the Input

Scikit-learn expects input features in two-dimensional format: (number of samples, number of features).

```
X = hours.reshape(-1, 1)
y = marks
```

```
print(X.shape)
print(y.shape)
```

Expected shapes: (8, 1) and (8,).

Step 3: Create and Train the Model

```
model = LinearRegression()
model.fit(X, y)
```

The fit() method learns the best-fitting regression line from the training data.

Step 4: Display Coefficient and Intercept

```
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
```

The regression equation is: Predicted Marks = coefficient × Hours + intercept.

Step 5: Predict the Existing Data

```
predicted = model.predict(X)
print(predicted)
```

Step 6: Predict a New Value

```
new_student = np.array([[6.5]])
prediction = model.predict(new_student)
print("Predicted Marks:", prediction[0])
```

Step 7: Evaluate the Model

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
mae = mean_absolute_error(y, predicted)
mse = mean_squared_error(y, predicted)
rmse = np.sqrt(mse)
r2 = r2_score(y, predicted)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R² Score:", r2)
```

5. Regression Evaluation Metrics

Mean Absolute Error (MAE)

$$\text{MAE} = (1/n) \sum |y - \hat{y}|$$

Represents the average absolute prediction error. Lower MAE indicates better predictions.

Complete Program

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

```
# Dataset
```

```

hours = np.array([1, 2, 3, 4, 5, 6, 7, 8])
marks = np.array([35, 40, 50, 55, 65, 70, 75, 85])

# Reshape input
X = hours.reshape(-1, 1)
y = marks

# Create model
model = LinearRegression()

# Train model
model.fit(X, y)

# Display model parameters
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)

# Predictions
predicted = model.predict(X)

# Evaluation metrics
mae = mean_absolute_error(y, predicted)
mse = mean_squared_error(y, predicted)
rmse = np.sqrt(mse)
r2 = r2_score(y, predicted)

print("\nModel Evaluation")
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

# New prediction
new_student = np.array([[6.5]])
prediction = model.predict(new_student)
print("\nPrediction for 6.5 hours:", prediction[0])

# Visualization
plt.scatter(hours, marks, label="Actual Data")
plt.plot(hours, predicted, label="Regression Line")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Linear Regression: Hours Studied vs Marks")
plt.legend()
plt.show()

```

Activity #3

Step 1: Change the Dataset

```

hours = np.array([2, 3, 4, 5, 6, 7, 8, 9])
marks = np.array([42, 48, 55, 61, 68, 74, 81, 88])

```

Train the model and calculate: coefficient, intercept, MAE, MSE, RMSE, and R^2 score.

Step 2: Prediction

Using your trained model, predict the marks for 3.5, 5.5, 7.5, and 10 hours. Create a NumPy array containing these values and predict all four simultaneously.

Activity #Challenge Activity

Consider the following dataset representing advertising expenditure and sales:

```
advertising = np.array([10, 20, 30, 40, 50, 60, 70, 80])
```

```
sales = np.array([25, 30, 38, 45, 50, 58, 65, 72])
```

Build a Linear Regression model to predict sales from advertising expenditure.

- Visualize the dataset.
- Train a Linear Regression model.
- Find the coefficient and intercept.
- Plot the regression line.
- Calculate MAE, MSE, RMSE and R^2 .
- Predict sales when advertising expenditure is 55.
- Predict sales for advertising expenditures of 25, 45 and 75.
- Interpret the coefficient in the context of advertising and sales.

8. Conclusion

In this experiment, Simple Linear Regression was implemented using Python. The relationship between an independent variable and a continuous dependent variable was analyzed, a regression line was fitted, predictions were generated, and model performance was evaluated using MAE, MSE, RMSE and R^2 score.